Homework 3 - Report

Álvaro Martínez Alfaro
Autonomous Robotics

April 16, 2026

1 Introduction

The objective of this assignment is to develop and integrate a trajectory-tracking control system for an autonomous agent operating within the *ROS2* `turtlesim` environment. The primary task requires the robot to continuously navigate a predefined closed-loop track, composed of two straightaways and two semi-circular curves with a radius of $r = 2$, at a constant linear velocity of $v_x = 0.2$ units per second. The controller governs the agent's heading by calculating the appropriate angular velocity (ω_z) to minimize the **cross-track error** (CTE) relative to the guidance path.

To achieve this, a **Proportional-Derivative** (PD) controller was implemented. The PD controller is a continuous feedback mechanism that computes an error value as the difference between the track center and the robot's current position, applying a restorative correction.

Formally, let cte_t denote the CTE at time t, defined as the orthogonal distance from the robot's origin to the target trajectory. The control law is defined as:

$$\delta_t = -\tau_p \cdot cte_t - \tau_d \cdot \frac{cte_t - cte_{t-1}}{\Delta t} \tag{1}$$

where τ_p generates a restorative force directly proportional to the current error magnitude and τ_d generates a control signal proportional to the instantaneous rate of change of the error.

This report details the implementation of this control architecture, focusing on the advanced development requirement: the dynamic optimization of the hyperparameters τ_p and τ_d by exploring the parameter space over consecutive laps to minimize the aggregate error.

2 Advanced development

The advanced implementation of this assignment focuses on two main improvements over the base controller: increasing the control loop frequency for greater stability and implementing a coordinate descent algorithm (**Twiddle**) to optimize the hyperparameters τ_p and τ_d.

2.1 Control loop discretization and frequency

The base implementation force the `listener_callback` to execute only once per second. This limitation was reduces to 0.1 seconds, allowing the robot to correct its orientation more frequently. The derivative term is computed using the actual elapsed time between callbacks:

$$\frac{dcte_t}{dt} \approx \frac{cte_t - cte_{t-1}}{\Delta t} \tag{2}$$

This ensures that the force applied by the τ_d term scales relatively to the physical time step, preventing the controller from becoming unstable at higher frequencies.

2.2 Cross-Track Error (CTE) computation

To provide accurate feedback to the controller, the track is partitioned into four regions. The CTE is calculated differently depending on the robot's spatial coordinates (x, y):

1. **Top straightaway**: for $x \in [3.5, 7.5]$ and $y > 5.5$, the reference trajectory is the horizontal line $y_{ref} = 7.5$. The error is calculated as $CTE = y - y_{ref}$.
2. **Bottom straightaway**: for $x \in [3.5, 7.5]$ and $y \leq 5.5$, the reference is $y_{ref} = 3.5$, and the error is $CTE = y_{ref} - y$.
3. **Left curve**: for $x < 3.5$, the reference is a semicircle centered at $(3.5, 5.5)$ with $r = 2.0$. Using the *Euclidean* distance metric, $CTE = \sqrt{(x - 3.5)^2 + (y - 5.5)^2} - r$.
4. **Right curve**: for $x > 7.5$, the reference is a semicircle centered at $(7.5, 5.5)$ with $r = 2.0$, and the error is $CTE = \sqrt{(x - 7.5)^2 + (y - 5.5)^2} - r$.

These calculations ensure that a positive CTE indicates the robot is outside the track, while a negative CTE indicates it has drifted inside the track.

2.3 Hyperparameter optimization

Instead of relying on static values for τ_p and τ_d, the system utilizes the **Twiddle** algorithm to search the parameter space for an optimal configuration. Twiddle is a local search optimization algorithm that iteratively adjusts one parameter at a time and evaluates the resulting system performance.

The cost function to be minimized is the Average Absolute CTE per lap, defined as:

$$J(\tau_p, \tau_d) = \frac{1}{N} \sum_{i=1}^{N} |cte_i| \tag{3}$$

where N is the number of callbacks processed during a lap. A lap is considered complete when the robot crosses the vertical midline at $x = 5.5$ in the positive direction within the top straightaway, guarded by a 2-second cooldown to prevent possible redundant triggers.

Furthermore, the Twiddle algorithm maintains a tuning vector $p = [\tau_p, \tau_d]$ and a step vector Δp. After evaluating the baseline error on the first lap, the algorithm enters a continuous state machine at the end of every lap, adjusting the parameters as follows:

1. **Exploration** (state 0): The algorithm increases the parameter space by adding Δp_i to the current parameter p_i. If $J(p)$ improves, the modification is kept, the step size is increased ($\Delta p_i \leftarrow 1.1 \cdot \Delta p_i$) to accelerate convergence, and the algorithm moves to the next parameter. Otherwise, if the error increases, the algorithm subtracts $2 \cdot \Delta p_i$ to probe the opposite direction and shifts to state 1.

2. **Reversion** (state 1): if the opposite direction yields a lower error, the change is kept, and the step size is increased. If both directions result in a higher error than the historical best, the parameter is reverted to its original state, and the step size is reduced ($\Delta p_i \leftarrow 0.9 \cdot \Delta p_i$) to refine the search resolution.

To visualize the optimization process, the system leverages an aynchronous ROS service call (`/turtle/set_pen`) to change the robot's trajectory color at the start of a new lap. It is asynchronous to avoid blocking the control loop.

3 Results

To evaluate the efficacy of the dynamic parameter optimization, the simulation was executed for 15 consequtive laps. The robot's initial spatial configuration was set to $(x_0, y_0, \theta_0) = (3.5, 7.5 \pm 0.5, r)$, where r is a random angle in the range $[-0.2, 0.2]$.

3.1 Baseline configuration

The controller was initialized with the baseline parameters $\tau_p = 0.4$ and $\tau_d = 0.6$. To initialize the local search, the Twiddle step sizes were set to $\Delta p_p = 0.05$ and $\Delta p_d = 0.05$. During the first lap, the system operated strictly with these parameters to establish a performance baseline, yielding an average absolute CTE of $J_{baseline} = 0.1716$.

3.2 Analysis of convergence

Following the initial lap, the descent algorithm activated. The Table 1 illustrates the parameter space exploration and the corresponding reduction in the objective function J over time.

As observed in Figure 1, the Twiddle algorithm achieved a near-monotonic convergence, reducing the average CTE from an initial baseline of approximately 0.17 down to under 0.08 by the final lap.

The parameter space search (Figure 1, bottom) reveals a coupled dynamic between the control gains. The algorithm consistently increased τ_p to minimize lateral drift along the

Table 1: Evolution of PD parameters and average error per lap via Twiddle optimization.

Lap	**Average CTE** ($\lvert cte_t \rvert$)	**Proportional Gain** (τ_p)	**Derivative Gain** (τ_d)
1 (Base)	0.1716	0.4000	0.6000
2	0.1560	0.4500	0.6500
3	0.1537	0.5050	0.6500
4	0.1378	0.5050	0.7050
5	0.1358	0.5655	0.7050
6	0.1219	0.5655	0.7655
7	0.1202	0.6321	0.7655
8	0.1081	0.6321	0.8321
9	0.1068	0.7053	0.8321
10	0.0961	0.7053	0.9053
11	0.0951	0.7858	0.9053
12	0.0860	0.7858	0.9858
13	0.0851	0.8744	0.9858
14	0.0770	0.8744	1.0744
Final	0.0763	0.9718	1.0744

straightaways. Because unilaterally increasing τ_p typically induces oscillations, the algorithm also increased τ_d to provide additional damping. This coupled stair-step growth allowed the controller to track the reference trajectory while mitigating overshoot in curved sections. Furthermore, the lack of downward adjustments in the parameter values indicates a smooth descent towards a local minimum without being destabilized.

4 Conclusion

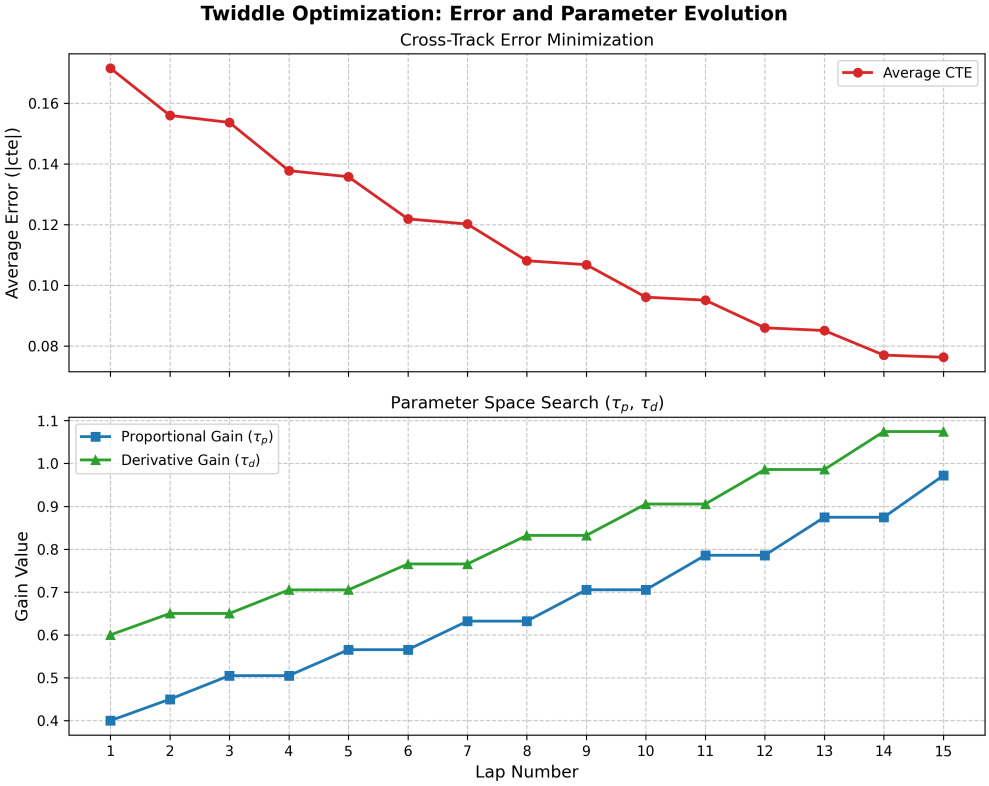

Figure 1: Convergence of the average CTE over consecutive laps.